

6.11.1 — Flight Protocol Structure + Actual RSC Payload Anatomy

Perfect. Now we're going to look **under the hood** at the thing we've been calling the RSC Payload.

You don't need to memorize the exact internal syntax. The goal is to understand **what each kind of record is communicating** and why React needs it.

1. First, the big picture

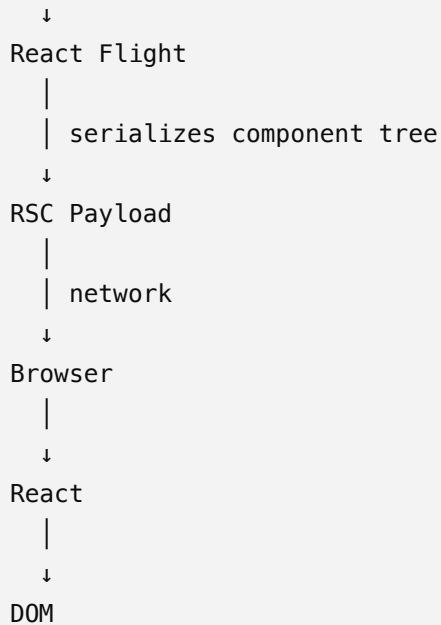
We currently have:

Server

|

| React Server Components





But the RSC Payload isn't one giant object.

It's more useful to think of it as a **stream of records/chunks**.

Conceptually:

```
graph TD; A[RSC Payload] --> B[record 1]; A --> C[record 2]; A --> D[record 3]; A --> E[record 4]; A --> F[record 5]; A --> G[...]
```

The diagram shows 'RSC Payload' at the top, with a vertical line extending downwards. From this line, five horizontal lines branch out to the right, each labeled 'record 1', 'record 2', 'record 3', 'record 4', and 'record 5' respectively. A final horizontal line branches out to the right, labeled '...'.

These records communicate different pieces of information.

2. Why chunks/records?

Imagine this Server Component:

```
export default async function Page() {
  const products = await getProducts();

  return (
    <main>
      <h1>Products</h1>
      <ProductList products={products} />
    </main>
  );
}
```

```
);  
}
```

The server might need to deal with:

```
Page  
↓  
getProducts()  
↓  
waiting...  
↓  
products arrive  
↓  
render tree  
↓  
Client Component reference
```



React doesn't necessarily need to wait for **every single piece** before sending anything.

Flight is designed to work with **streaming**.

So conceptually:

```
Server  
|  
├─ chunk 1 ──────────> Browser  
|  
├─ chunk 2 ──────────> Browser  
|  
├─ chunk 3 ──────────> Browser  
|  
└─ chunk 4 ──────────> Browser
```



The browser can process the Flight stream as it arrives.

This connects directly to the **Suspense/streaming** discussion we had earlier. 🔥

3. What does a Flight record represent?

Let's use a **fake simplified format** first.

Suppose the component tree is:

```
<main>  
  <h1>Hello</h1>
```

```
<ProductList products={products} />
</main>
```

Conceptually Flight might need information like:

```
1: main
2: h1 "Hello"
3: ClientModule "ProductList"
4: products [...]
5: element relationships
```



Again:

⚠️ These are NOT actual Flight bytes.

I'm deliberately simplifying them so you understand the architecture before seeing the ugly internal format.

4. The important categories of information

At a high level, Flight needs to communicate things such as:

A. React tree / elements

Something like:

```
main
├─ h1
│   └─ "Products"
└─ ProductList
```



B. Serialized values

For example:

```
products = [
  { id: 1, name: "Dosa" },
  { id: 2, name: "Pizza" }
]
```



These values can cross the Server → Client boundary when they're supported by React's serialization rules.

C. Client references

Something like:

```
"ProductList comes from client module X"
```



The browser then connects that reference to the Client Component JavaScript it has available.

D. Server references

For Server Functions / Server Actions, Flight can represent references that point back to server-side functionality.

Conceptually:

```
Server Function
```

```
↓
```

```
Reference ID
```

```
↓
```

```
Flight data
```

```
↓
```

```
Browser
```



E. Async / streamed results

React can represent work that hasn't resolved yet and continue streaming.

This is where Suspense becomes particularly interesting.

5. Let's look at a fake payload

Suppose:

```
export default function Page() {  
  return (  
    <main>  
      <h1>Products</h1>  
      <ProductList />  
    </main>  
  )  
}
```

```
);  
}
```

Imagine a **simplified fake Flight payload**:

```
1:"main"  
2:"h1"  
3:"Products"  
4:"$ClientReference:ProductList"  
5:["$1","$2","$3","$4"]
```



Don't memorize this syntax!

Instead, interpret it:

```
1 → main  
  
2 → h1  
  
3 → "Products"  
  
4 → Client Component reference  
  
5 → these things form the tree
```



So Flight is effectively communicating:

"Here's the React tree, here's its values, and here's where client-side code needs to take over."

6. Why are there weird \$ symbols?

When you eventually inspect a real RSC response, you'll see things that look **nothing like normal JSON**.

For example, you may encounter representations involving things like:

```
$L...  
$I...  
$...
```



or module/reference-related records.

The exact format is an **implementation detail of React Flight** and can change between React/Next.js versions.

The important thing is that special markers allow React to say:

"This isn't just a normal string."



"This refers to another record."

"This represents a Client Component."

"This is a special serialized value."

"This represents something asynchronous."

So don't look at the payload and think:

"Why isn't this JSON?" 😂

That's precisely because **Flight isn't simply JSON serialization.**

7. Let's connect this to Client Components

Consider:

```
// Server Component

import ProductList from './ProductList';

export default function Page() {
  return (
    <main>
      <ProductList />
    </main>
  );
}
```

And:

```
'use client';

export default function ProductList() {
  return <button>Add</button>;
}
```

The server needs to communicate:

There is a Client Component here.



But it doesn't need to send:

```
function ProductList() {  
  return <button>Add</button>;  
}
```

inside the RSC payload.

Instead, conceptually:

```
RSC Payload  
  |  
  ↓  
Client Reference  
  |  
  ↓  
"ProductList module/export"  
  |  
  ↓  
Browser's client module  
  |  
  ↓  
ProductList()
```



That's why the distinction is:

```
RSC Payload  
  ≠  
Client JavaScript bundle
```



They work together.

8. Now let's connect this to the Network tab 🔍

This is where the theory becomes REAL.

Suppose you have:

```
<Link href="/products/123">  
  Product 123
```

```
</Link>
```

You are currently on:

```
/products
```



You click the link.

Instead of thinking:

```
Browser
```

```
↓
```

```
GET complete HTML document
```

```
↓
```

```
replace everything
```



think:

```
Browser
```

```
↓
```

```
Next.js navigation request
```

```
↓
```

```
Server
```

```
↓
```

```
React renders new Server Component tree
```

```
↓
```

```
Flight
```

```
↓
```

```
RSC payload
```

```
↓
```

```
Browser
```

```
↓
```

```
React updates tree
```

```
↓
```

```
DOM
```



9. What you may see in DevTools

Open:

Chrome → DevTools → Network

Then navigate using a Next.js `<Link>` .

Depending on the Next.js version, request headers/URL parameters can expose that the request is asking for an **RSC response**.

You'll often see indicators such as:

```
RSC
Next-Router-State-Tree
Next-Router-Prefetch
```



The exact headers and payload presentation can vary across Next.js versions.

But the important observation is:

```
Navigation
  ↓
RSC request
  ↓
Flight/RSC response
```



10. And THIS is where Router Cache enters

Remember the Router Cache we learned?

The client-side Router Cache stores **RSC payload segments**.

So:

```
Server
  |
  | Flight / RSC payload
  ↓
Browser
  |
  ↓
Router Cache
  |
  ↓
React
  |
  ↓
DOM
```



This is why saying:

"Router Cache stores pages"

isn't quite precise enough.

A better mental model is:

Router Cache stores the RSC payload/tree information for visited or prefetched route segments on the client.

11. Example: first visit

You navigate:

/products



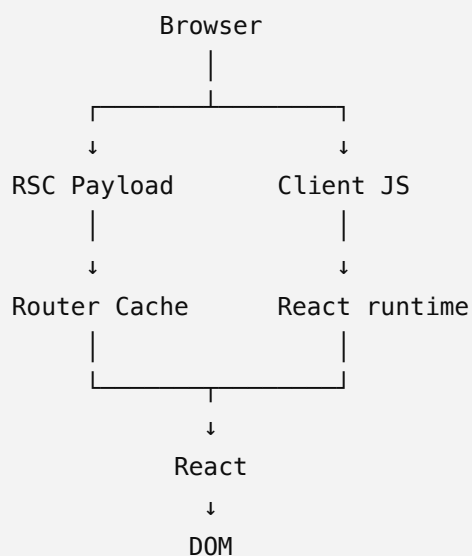
Server generates:

RSC payload



Browser receives it.

Conceptually:



12. Then you navigate to /products/123

Server generates the new RSC result.

/products
↓
click
↓
/products/123
↓
RSC request



↓
new Flight payload

React receives the new payload.

It doesn't blindly throw away the entire application tree.

It can reconcile the new Server Component result with the existing tree.

Conceptually:

OLD TREE

Root

├─ Header
└─ Products

NEW TREE

Root

├─ Header
└─ ProductDetails

React can preserve/reuse what remains relevant:

Header

↑
may remain

Products

↑
removed/replaced

ProductDetails

↑
new

That's the magic behind App Router navigation.

13. Now let's connect this to layouts

Suppose:

app/
├─ layout.tsx
└─ products/

```

├─ layout.tsx
├─ [id]/
│   └─ page.tsx

```

When moving:

```

/products
  ↓
/products/123

```



the root layout and products layout may remain the same.

Only the relevant segment changes.

Conceptually:

```

Root Layout
├─ Products Layout
│   └─ Page
└─ ...
    ↓
RSC tree

```



The navigation payload can therefore represent the changed portion of the tree.

And this is **one of the reasons nested layouts + RSC + Router Cache work so nicely together.**

14. Now let's connect prefetching

Suppose you have:

```
<Link href="/products/123">
```

Next.js can prefetch the destination in situations where prefetching applies.

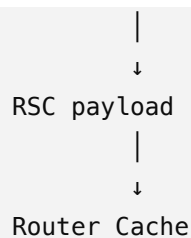
Conceptually:

```

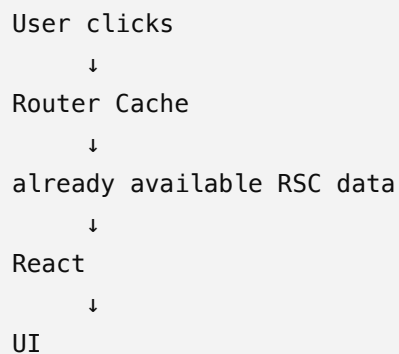
User hasn't clicked yet
  ↓
Next.js prefetches

```





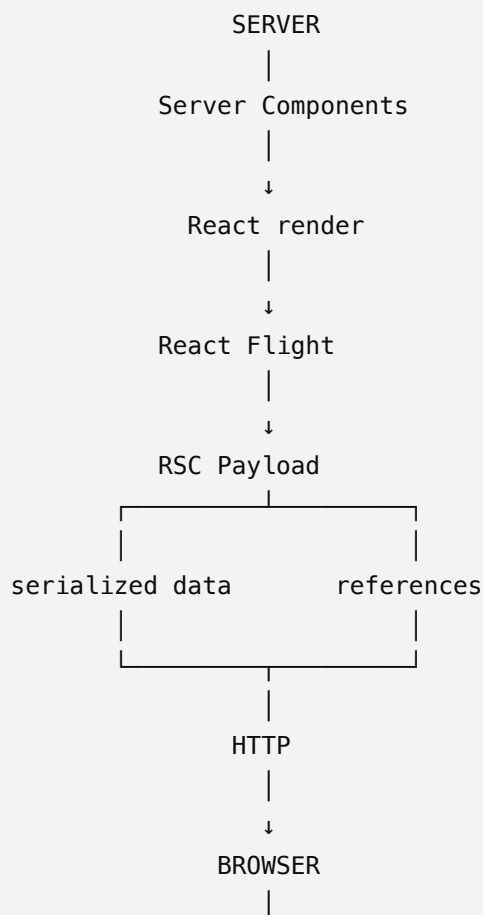
Then:

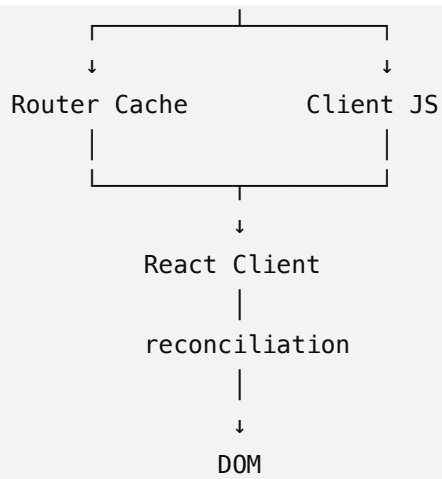


So the navigation can feel extremely fast.

15. Now the complete pipeline

This is the diagram I want you to keep. 🧠





16. And for navigation

User clicks <Link>



Client Router

Router Cache hit?
 YES → use cached RSC data
 NO

request server

React Server Components

React Flight

RSC Payload

Router Cache

React reconciliation

DOM update



This is the connection between the things you've been learning separately.

17. One subtle but VERY important correction

Don't think:

RSC Payload = HTML



or:

RSC Payload = JSON



or:

RSC Payload = JavaScript bundle



They're different:

Thing	Main purpose
HTML	Initial browser document/UI
RSC Payload	Serialized Server Component tree/data/references
Client JS bundle	Executable Client Component code + client React code
Router Cache	Client-side cached RSC data/tree segments
DOM	Actual UI structure currently displayed

And:

Flight
↓
protocol/format/mechanism

RSC Payload
↓
actual data produced using Flight



The interview version

If they ask:

"What happens during an App Router navigation?"

A strong answer is:

"Next.js performs a client-side navigation and requests the Server Component result rather than doing a full document navigation. React Server Components are rendered on the server and serialized using the Flight protocol into an RSC payload. The browser receives that payload, combines it with the existing client-side React tree and Client Component code, and React reconciles the result to update the DOM. The RSC data can also be stored in the client Router Cache for subsequent navigation or reuse."

That's the whole chain in one answer.

One thing I want you to notice 🧐

We've now reached an important junction:

```
Flight
  ↓
RSC Payload
  ↓
serialization
  ↓
Client References
  ↓
Server References
  ↓
Server Actions
```



Next: 6.11.2 — Serialization in depth.

There we'll answer the question that naturally comes next:

"Exactly what CAN and CANNOT cross the Server → Client boundary, and why?"

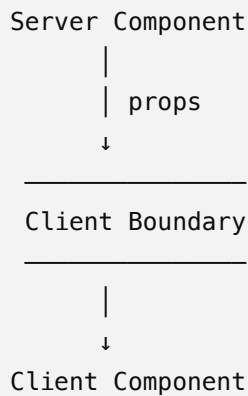
We'll go through **objects, arrays, Dates, Maps/Sets, Promises, functions, Server Actions, Client Components**, and why `'use client'` creates a serialization boundary. 🚀



okay

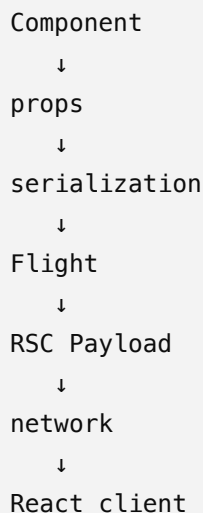
😊 Perfect. We'll continue with **6.11.2 — Serialization in Depth** next.

We'll build it from first principles:



And answer:

1. What exactly is serializable?
2. Why can objects/arrays cross the boundary?
3. What about **Date, Map, Set, Promise**?
4. Why can't an ordinary function cross?
5. Why **Server Actions are special**?
6. What exactly does `'use client'` create?
7. How Client References and Server References fit into serialization.
8. Finally, we'll inspect the whole thing as:



This is the piece that will make the **Server/Client boundary** really click. 🚀

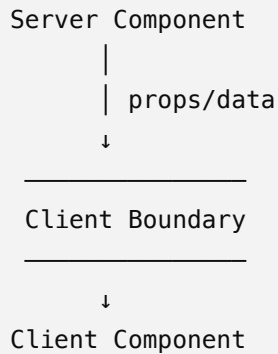


sure go ahead

6.11.2 — Serialization in Depth

Let's gooo 😄. This is one of those topics where a small distinction makes the whole Server/Client architecture suddenly make sense.

We already know:



The question is:

What is actually allowed to cross that boundary?

1. First: What does "serialization" mean here?

Forget React for a second.

Suppose the server has this JavaScript value:

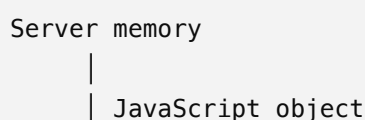
```
</> JavaScript

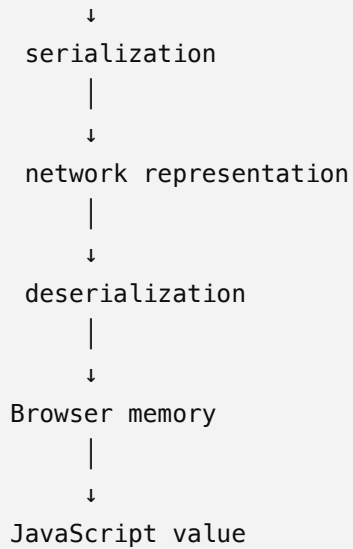
const user = {
  id: 42,
  name: "Gopi"
};
```

Inside the server, that's an actual JavaScript object in memory.

The browser cannot directly access that memory.

So if we want to send its information over the network, we need to represent it in a transferable form:





That's the basic idea.

2. In RSC, it's NOT simply JSON

This distinction is important.

You might think:

```
</> JavaScript  
JSON.stringify(user)
```



But React Flight has its own serialization mechanism.

Why?

Because React needs to communicate more than ordinary JSON data.

For example:

```
normal values  
React elements  
Client References  
Server References  
Promises  
special values  
etc.
```



So:

```
JSON serialization  
✗
```



Flight serialization



Think of Flight as having a richer vocabulary.

3. Simple values

These are the easy ones.

Suppose:

`</>` TypeScript



```
<Product
  name="Dosa"
  price={80}
  available={true}
/>
```

Conceptually:

name
↓
"Dosa"



price
↓
80

available
↓
true

These values can cross the Server → Client boundary.

So:

Server



```
|
├─ string
├─ number
├─ boolean
├─ null
|
↓
Client
```

No mystery here.

4. Objects and arrays

Suppose the Server Component does:

```
</> TypeScript   
  
export default async function Page() {  
  const product = {  
    id: 10,  
    name: "Masala Dosa",  
    price: 80  
  };  
  
  return <ProductCard product={product} />;  
}
```

And:

```
</> TypeScript   
  
'use client';  
  
function ProductCard({ product }) {  
  return <h2>{product.name}</h2>;  
}
```

The object can cross the boundary.

Conceptually:

SERVER 

```
product  
{  
  id: 10,  
  name: "Masala Dosa",  
  price: 80  
}
```

|
| Flight serialization
↓

BROWSER

```
product
```

```
{
  id: 10,
  name: "Masala Dosa",
  price: 80
}
```

Notice something important:

The **same JavaScript object identity** isn't being transferred.

The information is transferred and represented on the client.

So don't imagine:

Server object —————> exact same object —————> Browser



Instead:

```
Server object
  ↓
serialized representation
  ↓
network
  ↓
client-side value
```



5. Nested objects work too

For example:

```
</> TypeScript

<ProductCard
  product={{
    id: 1,
    name: "Dosa",
    restaurant: {
      name: "BhojanaLaya",
      city: "Bangalore"
    }
  }}
/>
```



Conceptually:

```
product
├── id
├── name
└── restaurant
    ├── name
    └── city
```



The nested structure can be represented by Flight.

6. Arrays

Same idea:

</> TypeScript



```
<ProductList
  products=[
    { id: 1, name: "Dosa" },
    { id: 2, name: "Idli" },
    { id: 3, name: "Vada" }
  ]
/>
```

The array and its supported contents can cross.

Server

```
|
| [
|   {...},
|   {...},
|   {...}
| ]
```

↓

Flight

↓

Client



7. Now the interesting ones: Date, Map, Set

Here's where you shouldn't use the simplistic rule:

"Only JSON values are serializable."

That's **too simplistic for React Flight**.

React's RSC serialization supports more types than plain JSON.

For example, React's supported serialization model includes types such as:

Date
Map
Set
Promise



among other supported values.

So conceptually:

```
</> TypeScript

<Product
  createdAt={new Date()}
  tags={new Set(["food", "south-indian"])}
/>
```



is not equivalent to saying:

```
"JSON.stringify everything"
```



Flight has special handling for supported values.

8. Why does Flight need special handling?

Because JavaScript has types that JSON doesn't preserve properly.

For example:

```
</> JavaScript

const date = new Date();
```



If you use JSON:

```
</> JavaScript

JSON.stringify(date)
```



you don't get a Date object back automatically.

You get a string representation.

Likewise, JSON doesn't natively represent:

Map
Set
Promise
undefined



in the same way JavaScript does.

Flight therefore has its own encoding conventions.

That's why we keep saying:

Flight serialization $\neq$ JSON.stringify().

9. Promises are especially interesting 🍹

Suppose:

```
</> TypeScript

const productsPromise = getProducts();

return (
  <ProductList products={productsPromise} />
);
```



React Server Components can work with Promises as part of their streaming model.

Conceptually:

```
Promise
  |
  | not resolved yet
  ↓
Flight
  |
  ↓
Browser receives reference/result
  |
  ↓
Promise resolves
  |
  ↓
React continues rendering
```



This is closely connected to the **Suspense + streaming** concepts we studied earlier.

That's why these topics aren't isolated:

```
Promise
  ↓
Suspense
  ↓
Streaming
  ↓
Flight
  ↓
RSC Payload
```



It's all one system.

10. Now the big NO: ordinary functions

Suppose you have:

```
</> TypeScript

function sayHello() {
  console.log("hello");
}
```



And:

```
</> TypeScript

<ClientButton onClick={sayHello} />
```



where:

```
</> TypeScript

'use client';
```



The server cannot just serialize:

```
</> JavaScript

function sayHello() {
  console.log("hello");
}
```



and send the function implementation to the browser as an ordinary prop.

Why?

Because a function isn't merely data.

It contains executable behavior and is tied to an execution environment.

The server's:

```
sayHello()
```



belongs to the server's JavaScript environment.

The browser can't simply receive the server's function object and execute it there.

So:

Server function



ordinary serialization



11. Then how does `onClick` normally work?

This is a VERY important distinction.

You can absolutely have:

`</>` TypeScript



```
'use client';

function ClientButton() {
  function sayHello() {
    console.log("hello");
  }

  return (
    <button onClick={sayHello}>
      Click
    </button>
  );
}
```

This works because:

```
sayHello
  ↓
created inside Client Component
  ↓
client JavaScript
  ↓
browser
```



The function never had to cross the Server → Client boundary.

So:

✗ Server function → Client prop

```
Server function
  ↓
Client Component
```



Not an ordinary serializable prop.

✓ Client function → Client Component

```
Client Component
  ↓
function
  ↓
browser
```



Perfectly normal.

12. Now the special case: Server Functions

Here's where things get cool.

Suppose:

```
</> TypeScript

'use server';

export async function addToCart(productId) {
  await db.cart.add(productId);
}
```



This function **stays on the server**.

But React/Next.js can create a **reference** to it.

Think:

Actual function:

addToCart()



Server Reference

"function ID = XYZ"



The browser doesn't receive the implementation.

It receives enough information to say:

"I want to invoke Server Function XYZ."

So:

Browser



invoke reference XYZ

Server



addToCart()



13. This is why Server Actions aren't magic function teleportation

This:

`</> TypeScript`

`<form action={addToCart}>`

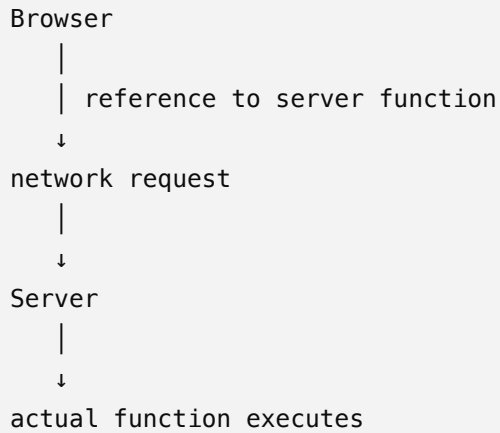


doesn't mean:

Browser somehow possesses addToCart()



It means something closer to:



This is **Server Reference** territory.

14. Client References are the opposite

Now let's flip direction.

You have:

```
</> TypeScript

'use client';

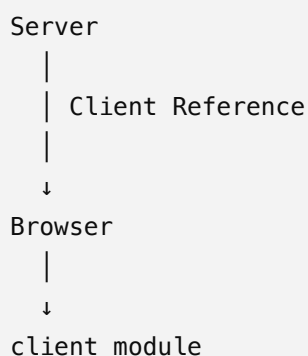
export function ProductList() {
  ...
}
```

The Server Component tree needs to say:

"There is a Client Component here."

So Flight can represent a **Client Reference**.

Conceptually:



↓
ProductList

Again:

Client Reference
≠
actual component code



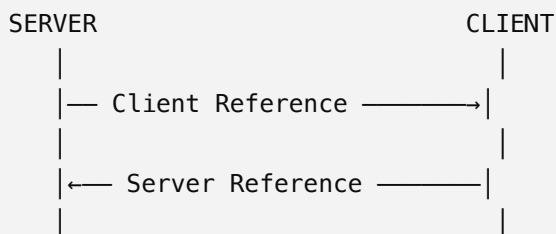
It's a reference to the client module/export.

15. So now we have two special references

This is worth memorizing:

Reference	Direction	What it represents
Client Reference	Server → Client	Reference to Client Component/module
Server Reference	Client → Server	Reference to Server Function

Visual:



This is a beautiful mental model.

16. What exactly does 'use client' do?

This is another place people misunderstand things.

When you write:

```
</> TypeScript  
'use client';
```



you aren't saying:

"This component will only ever execute in the browser."

It's more nuanced.

You're essentially declaring a **Client Component boundary/module graph**.

That tells the framework:

This module belongs to the client side.
Its dependencies become part of the client bundle graph.



And importantly, props crossing into that boundary must follow the supported serialization model.

So:

```
Server Component
  |
  | serializable props
  ↓
'use client' boundary
  |
  ↓
Client Component
```



17. Here's the classic mistake

People say:

"`use client` means this component is rendered only in the browser."

✗ Not a good definition.

Instead:

`'use client'` marks a module as a **Client Component entry point**, establishing a **boundary between the server and client component graphs**.

The Client Component can participate in server rendering of the initial UI, but its client-side JavaScript is also made available for browser interactivity.

That's a much better interview answer.

18. Let's trace a complete example

Suppose:

```
</> TypeScript   
  
// Server Component  
  
export default async function Page() {  
  const product = {  
    id: 1,  
    name: "Dosa",  
    price: 80  
  };  
  
  return (  
    <ProductCard product={product} />  
  );  
}
```

Client:

```
</> TypeScript   
  
'use client';  
  
export function ProductCard({ product }) {  
  return (  
    <button>  
      {product.name} - ₹{product.price}  
    </button>  
  );  
}
```

What happens?

Step 1

Server executes:

```
Page()
```



↓

```
product  
{  
  id: 1,  
  name: "Dosa",
```



```
price: 80  
}
```

Step 2

React encounters:

ProductCard



which is a Client Component.

Step 3

Flight needs to represent:

Client Reference → ProductCard



and:

product → serialized data



Step 4

RSC Payload travels:

```
Server  
  ↓  
Flight  
  ↓  
RSC Payload  
  ↓  
Browser
```



Step 5

Browser combines:

```
RSC information  
  +  
Client JS module  
  ↓  
React  
  ↓  
ProductCard
```



19. What if we try this?

</> TypeScript



```
<ProductCard
  product={product}
  onSave={saveProduct}
/>
```

where:

</> TypeScript



```
function saveProduct() {
  // server-side stuff
}
```

Now the question becomes:

What exactly is `saveProduct` ?

If it's an ordinary server function:

✗ Can't simply cross as an ordinary prop.



If it's a supported Server Function / Server Action:

✓ React can represent it as a Server Reference.



So the key isn't:

"Functions can never cross."

The more accurate rule is:

Ordinary functions cannot be passed across the Server → Client boundary as serializable props. Server Functions have special reference semantics that allow them to be invoked remotely.

That's the subtle answer.

20. Now connect EVERYTHING we've learned

We started with:

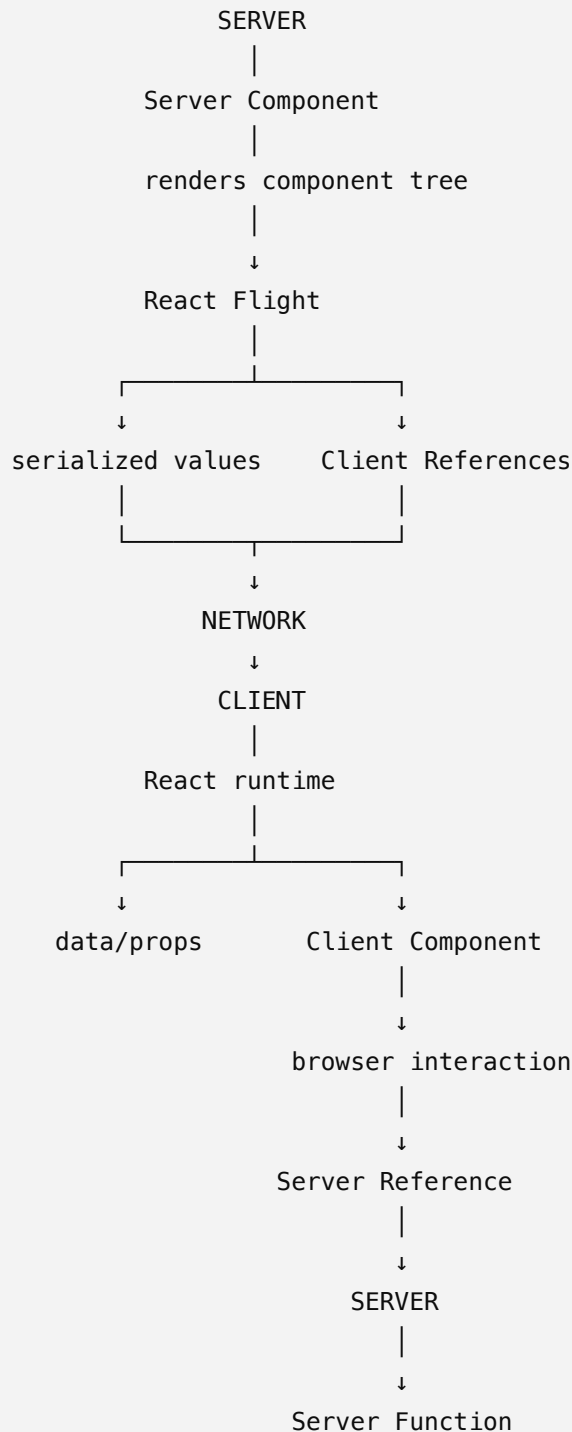
Server Component



Client Component



Now we can expand it:



The most important mental model

Don't think:

Server sends components to browser



Think:

Server executes Server Components



↓

React creates a representation of the result

↓

Flight serializes that representation

↓

RSC Payload travels to browser

↓

Client References point to client-side modules

↓

Client React reconstructs/reconciles the tree

And:

Server Component code



doesn't need to be shipped to browser

Client Component code



is included in client JS

Server Function



implementation isn't shipped

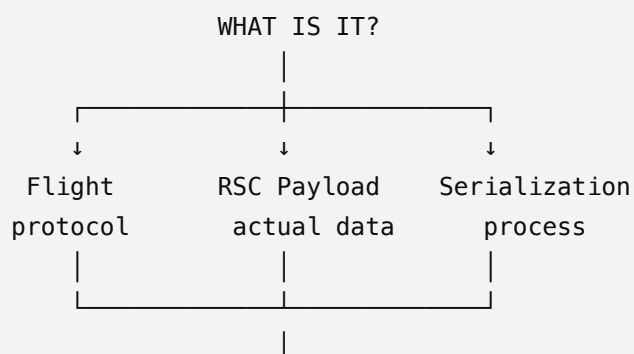
Server Reference

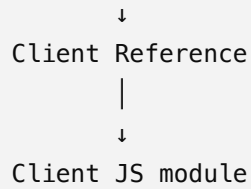


allows the browser to request its execution

One final distinction

These four things are easy to mix up:





And separately:

Server Function



Server Reference



"invoke this on server"



So if an interviewer asks:

"What is serialization in RSC?"

Say:

"Serialization is the process of converting the Server Component tree, supported prop values, and special references into Flight's transferable representation so React can send that information from the server to the client and reconstruct/reconcile the tree there. It's richer than plain JSON serialization because Flight also has semantics for React-specific values and references."

That's the level of understanding we're aiming for. 🚀





😊 Perfect. Next we'll do **6.11.7 — the complete RSC + React Flight recap and interview questions.**

After that, **6.11.x is officially done 🏁**, and we'll start **6.12 in a new chat.**



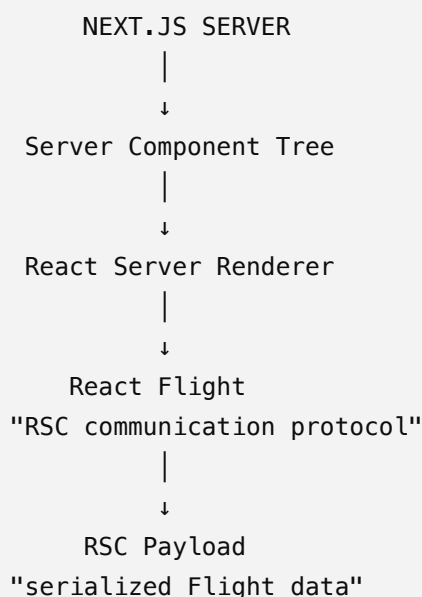
sure go ahead

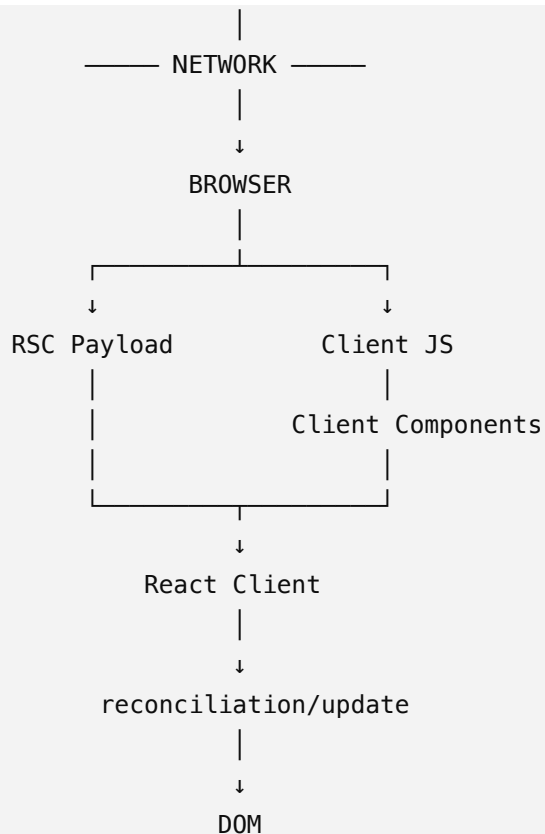
🏁 6.11.7 — Complete RSC Payload + React Flight Recap

Alright 😊 this is the **final 6.11.x lesson**. After this, we're done with RSC/Flight fundamentals and **6.12 will start in a fresh chat**, as planned.

Let's consolidate everything into one mental model first.

1. The entire RSC story in one picture





The biggest thing to remember:

Server Components don't need to be shipped to the browser as JavaScript. Their rendered result is represented in the RSC payload.

Client Components are different because their JavaScript needs to be available in the browser for interactivity.

2. Flight vs RSC Payload

This was one of your original questions.

React Flight

Think:

The protocol/mechanism used by React to transport Server Component information.

RSC Payload

Think:

The actual serialized data produced using that mechanism for a particular render/navigation.

So:

Flight

↓

protocol / mechanism

RSC Payload

↓

actual Flight data



Don't say:

"Flight is the payload."



Say:

"The RSC payload is transmitted using React Flight."



3. What does "serialized representation" mean?

Suppose the server has:

`</> TypeScript`

```
<Product
  name="Dosa"
  price={80}
/>
```



React has an internal representation of this component tree.

That internal representation can't simply be sent directly over HTTP.

So React converts the relevant information into a **transferable Flight representation**.

React tree

↓

serialization

↓

Flight representation

↓

network



↓
client React

So:

Serialized representation = a network-transferable representation of the Server Component tree, supported values, and relevant references.

And importantly:

Flight serialization
≠
JSON.stringify()



Flight has React-specific semantics.

4. Client References

Suppose:

```
</> TypeScript

'use client';

export default function AddButton() {
  return <button>Add</button>;
}
```



The Server Component tree can contain:

```
</> TypeScript

<AddButton />
```



The server doesn't need to send the component's source code inside the RSC payload.

Instead, Flight can represent a **Client Reference**.

Conceptually:

```
Server
|
| "Client Component X goes here"
↓
Client Reference
|
↓
```



```
Browser
|
↓
client JS module
|
↓
AddButton
```

So:

```
Client Reference
  ≠
Client Component source code
```

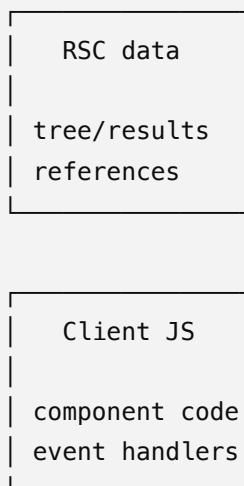


It's a reference to the client-side module/export.

5. Client JS vs RSC Payload

Don't mix these up.

BROWSER



The two work together.

RSC Payload

Answers roughly:

"What Server Component result/tree should React use?"

Client JS

Provides:

"Here is the executable code for Client Components."

6. Server Functions / Server References

Now reverse the direction.

```
</> TypeScript

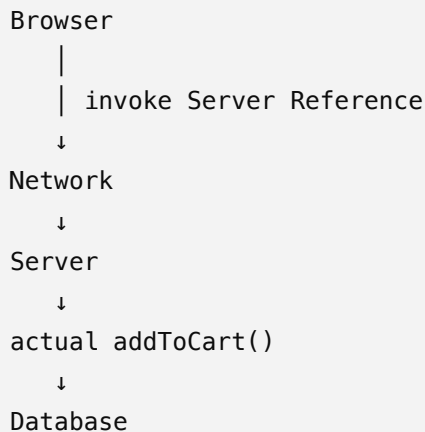
'use server';

export async function addToCart(productId) {
  await db.cart.add(productId);
}
```

The browser doesn't receive the implementation.

Instead, it can receive/use a **Server Reference**.

Conceptually:



And remember the important distinction:

function that happens to execute on server
≠
Server Function

A Server Function has explicit Server Function semantics, typically via:

```
</> TypeScript

'use server';
```

7. Ordinary function vs Server Function

Ordinary server-side function

</> TypeScript



```
function getProducts() {  
  return db.products.findMany();  
}
```

It may execute on the server.

But it isn't automatically remotely invokable.

server-side function

↓

runs on server

↓

ordinary function



Server Function

</> TypeScript



```
'use server';  
  
export async function addToCart() {  
  ...  
}
```

Server Function

↓

Server Reference

↓

can be invoked through supported server-action mechanisms



8. Serialization boundary

Suppose:

</> TypeScript



```
export default function Page() {  
  const product = {  
    id: 1,  
  }  
}
```

```

    name: "Dosa"
  };

  return <ProductCard product={product} />;
}

```

and:

```

</> TypeScript

'use client';

function ProductCard({ product }) {
  ...
}

```

The `product` data crosses:

```

Server
  |
  | serializable value
  ↓
Client boundary
  |
  ↓
Client Component

```

But an ordinary server function cannot simply be passed the same way:

```

</> TypeScript

<ProductCard
  onSave={someServerFunction}
/>

```

unless that function has the appropriate **Server Function/reference semantics**.

9. 'use client' vs 'use server'

Keep this distinction very clear.

'use client'

```

'use client'
  ↓
Client Component boundary
  ↓

```

```
client module graph
```

```
↓
```

```
browser JS
```

```
'use server'
```

```
'use server'
```

```
↓
```

```
Server Function semantics
```

```
↓
```

```
Server Reference
```

```
↓
```

```
remote invocation
```



And:

'use client' does **not** simply mean "this component only renders in the browser."

That's a common oversimplification.

10. RSC navigation

Now suppose:

```
/products
```



and:

```
</> TypeScript
```

```
<Link href="/products/123">
```



is clicked.

The important flow is:

```
<Link>
```

```
↓
```

```
Next.js Client Router
```

```
↓
```

```
RSC request
```

```
↓
```

```
Server Components execute/render
```

```
↓
```

```
React Flight
```

```
↓
```



```
RSC Payload
↓
Browser
↓
React reconciliation
↓
DOM update
```

Notice:

No full HTML document replacement is required for this client-side navigation.

11. DOM can change without replacing the document

This was another question you asked earlier.

Suppose:

Before

```
<main>
  <h1>Products</h1>
  <ProductList />
</main>
```



After navigation

```
<main>
  <h1>Product 123</h1>
  <ProductDetails />
</main>
```



The DOM can change substantially.

But:

```
DOM changed
  ≠
whole HTML document replaced
```



React can update the existing DOM based on the new RSC result.

That's a **very important App Router concept**.

12. Router Cache

The RSC payload received during navigation can be used by the App Router and stored in its client-side Router Cache.

Conceptually:

```
Server
  ↓
RSC Payload
  ↓
Router Cache
  ↓
React
  ↓
DOM
```



Router Cache is:

Client-side cached RSC route/segment information used by the App Router.

Don't confuse it with:

```
Data Cache
Full Route Cache
browser HTTP cache
JS bundle cache
```



They're different things.

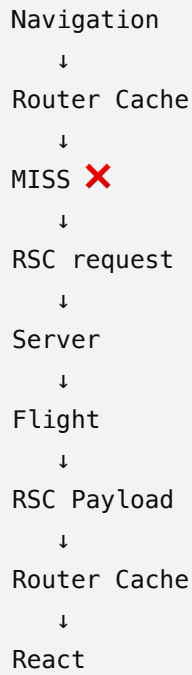
13. Cache HIT vs MISS

HIT

```
Navigation
  ↓
Router Cache
  ↓
HIT ✓
  ↓
reuse cached RSC information
  ↓
React
  ↓
DOM
```



MISS



```
graph TD; Navigation --> RouterCache1[Router Cache]; RouterCache1 --> MISS[MISS ❌]; MISS --> RSCrequest[RSC request]; RSCrequest --> Server; Server --> Flight; Flight --> RSCPayload[RSC Payload]; RSCPayload --> RouterCache2[Router Cache]; RouterCache2 --> React;
```

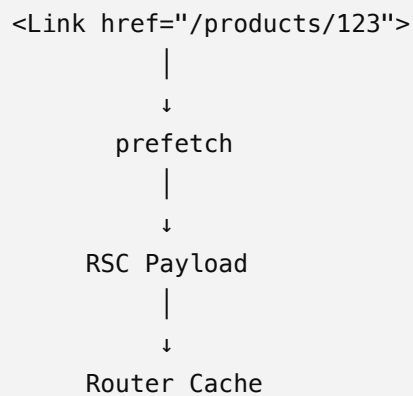
Navigation
↓
Router Cache
↓
MISS ❌
↓
RSC request
↓
Server
↓
Flight
↓
RSC Payload
↓
Router Cache
↓
React

This is the core navigation-caching model.

14. Prefetching

A `<Link>` can also be prefetched in applicable situations.

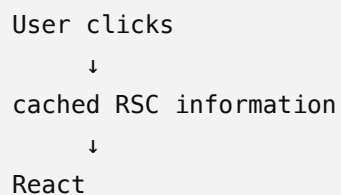
Conceptually:



```
graph TD; Link["<Link href='/products/123'>"] --> prefetch; prefetch --> RSCPayload[RSC Payload]; RSCPayload --> RouterCache[Router Cache];
```

`<Link href="/products/123">`
↓
prefetch
↓
RSC Payload
↓
Router Cache

Then:



```
graph TD; UserClicks[User clicks] --> cachedRSC[cached RSC information]; cachedRSC --> React;
```

User clicks
↓
cached RSC information
↓
React

↓
UI

So a request can happen **before the user actually navigates**.

15. Streaming + Suspense

Now connect the last part.

Suppose:

Header → fast
Products → fast
Reviews → slow



With:

```
</> TypeScript

<Suspense fallback={<ReviewsSkeleton />}>
  <Reviews />
</Suspense>
```



React can conceptually produce:

Header
↓
Products
↓
ReviewsSkeleton



then later:

Reviews
↓
replace fallback



Flight allows the RSC result to be **streamed progressively**.

So:

Suspense
↓
defines boundary

Streaming



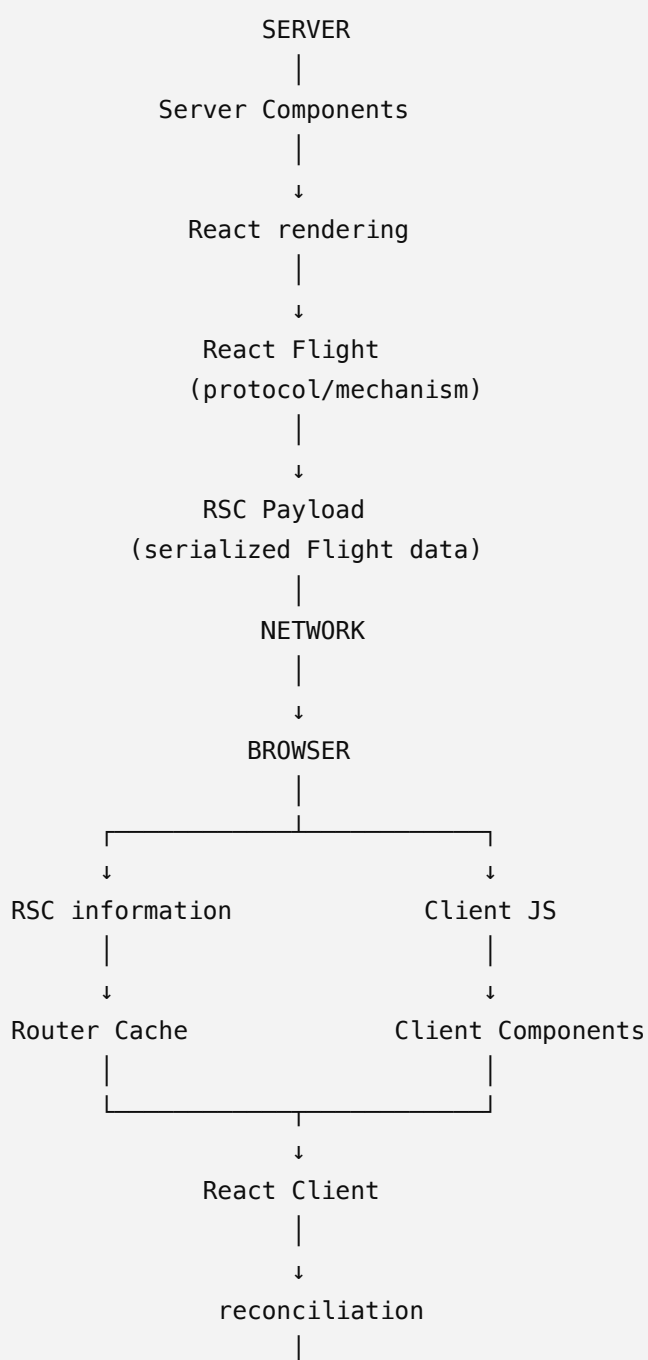
↓
delivers results progressively

Flight
↓
transports RSC representation

These aren't interchangeable concepts.

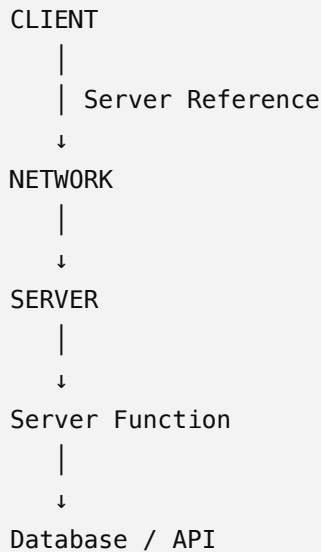
16. The complete architecture 🔥

Here's your final 6.11 mental model:



↓
DOM

For server mutations:



🎯 17. Interview Questions

Now let's test whether you've actually got it rather than just nodding along. 😂

Q1. What is React Flight?

Answer:

React Flight is the protocol/mechanism React uses to communicate the Server Component tree between server and client.

Q2. What is the RSC Payload?

The RSC Payload is the serialized Flight data representing the Server Component result/tree, supported values, and relevant references for a particular render or navigation.

Q3. Is RSC Payload HTML?

No.

HTML

→ document/UI representation



RSC Payload

→ React Server Component representation

Q4. Is RSC Payload JSON?

Not simply.

Flight has its own serialization format and semantics beyond ordinary JSON serialization.

Q5. Does the browser receive Server Component source code?

No.

The Server Component executes on the server, and its result is represented in the RSC payload.

Q6. Does the browser receive Client Component JavaScript?

Yes.

Client Components need client-side JavaScript for their interactivity.

Q7. What is a Client Reference?

A reference in the RSC system to a Client Component/module that the browser can resolve to its client-side code.

Q8. What is a Server Reference?

A reference to a Server Function that allows the client to request its invocation without receiving the function implementation.

Q9. Can any function become a Server Function just because it runs on the server?

No.

```
runs on server
  ≠
Server Function
```



Server Function semantics are explicitly established, typically with `'use server'`.

Q10. During `<Link>` navigation, does Next.js normally download a complete new HTML document?

No.

For client-side App Router navigation, Next.js can request the RSC payload and use it to update the existing React tree and DOM.

Q11. Can the DOM structure change during client navigation?

Absolutely.

```
existing DOM
  ↓
React receives new RSC result
  ↓
reconciliation
  ↓
new DOM structure
```



But the entire browser document doesn't have to be replaced.

Q12. What does Router Cache store?

Client-side RSC route/segment information used by the App Router for navigation and reuse.

18. Your 30-second interview answer

If they ask:

"Explain React Server Components and RSC Payload in Next.js."

You can say:

"Next.js uses React Server Components to execute server-side component logic. React's Flight protocol provides the mechanism for representing and transporting the Server Component result to the client as an RSC payload. The payload contains serialized component-tree information, supported values, and references to Client Components. The client combines that information with the Client Component JavaScript and React uses it to construct or reconcile the UI. During App Router navigation, the RSC payload can be fetched and stored in the client Router Cache, allowing the existing application tree to be updated without a full document reload."

That's a **very solid mid-level answer**. 🎯

🏁 6.11.x — COMPLETE

You now have:

6.11 RSC Payload + React Flight



- |
- |— Flight
- |— RSC Payload
- |— serialization
- |— Client References
- |— Server References
- |— Server Functions
- |— RSC navigation
- |— Router Cache relationship
- |— prefetching
- |— streaming
- |— Suspense

🏁 DONE

And now we **STOP** this chat. 🙅

As planned, **don't start 6.12 here**.

Open a **new chat** and say:

"Continue Next.js 6.12"

Then we'll begin the next major piece:

6.12 — Serialization, Client References & Server References

We'll go deeper there, without mixing it back into 6.11. 😊



server function vs server side function invocation and diff

Yes — this distinction is **super important**, because "server-side function" and "Server Function" sound almost identical. 😊

1. Server-side function

A **server-side function** simply means:

A normal function whose code executes on the server.

Example:

```
</> TypeScript

async function getProducts() {
  const products = await db.products.findMany();
  return products;
}

export default async function Page() {
  const products = await getProducts();

  return <ProductList products={products} />;
}
```

Here:

```
Page()
  ↓
getProducts()
  ↓
Database
```

`getProducts()` runs on the server.

But it's just a **normal function call**:

```
Page()  
  |  
  | normal JS call  
  ↓  
getProducts()
```



There is **no network request** between **Page** and **getProducts**.

The function isn't remotely callable by the browser.

2. Server Function

A **Server Function** is different.

It is explicitly given Server Function semantics, typically with:

```
</> TypeScript  
  
'use server';  
  
export async function addToCart(productId) {  
  await db.cart.add(productId);  
}
```



Now the function can be exposed through a **Server Reference**.

Conceptually:

```
Browser  
  |  
  | invoke Server Reference  
  ↓  
NETWORK REQUEST  
  ↓  
Server  
  ↓  
addToCart(123)
```



So there's a fundamental difference:

	Server-side function	Server Function
Runs on server	✓	✓

	Server-side function	Server Function
Normal JS function	✓	Also a function, but special semantics
'use server'	✗	✓
Callable remotely from client	✗	✓ through supported mechanisms
Server Reference	✗	✓
Network involved when called from server code	✗	Not necessarily
Designed for client → server invocation	✗	✓

3. "Server-side function invocation" means something different

Suppose:

```
</> TypeScript

async function getUser() {
  return db.user.find();
}

async function Page() {
  const user = await getUser();
}
```

The invocation is:

```
Page()
|
| function call
↓
getUser()
```

Everything is happening inside the **same server execution environment**.

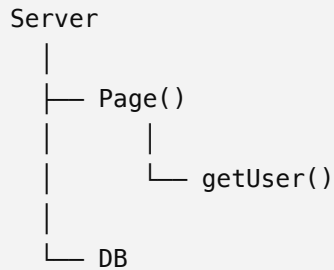
There's no:



```
graph TD; browser --> network; network --> server;
```

browser
↓
network
↓
server

It's simply:



```
graph TD; Server --> Page; Server --> DB; Page --> getUser;
```

Server
|
├ Page()
|
└ getUser()
|
└ DB

4. Server Function invocation is different

Now:



```
</> TypeScript

'use server';

export async function addToCart(productId) {
  await db.cart.add(productId);
}
```

And the browser wants to invoke it.

The browser doesn't have:

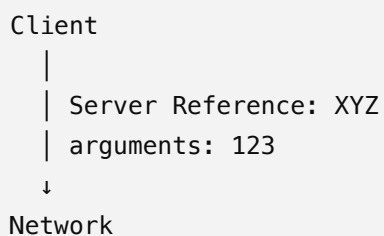


```
</> JavaScript

addToCart() // implementation
```

Instead it has a reference.

Conceptually:



```
graph TD; Client --> Network; Client --> ServerReference[Server Reference: XYZ]; Client --> Arguments[arguments: 123];
```

Client
|
| Server Reference: XYZ
| arguments: 123
↓
Network

```
↓  
↓  
Server  
|  
| resolve XYZ  
↓  
addToCart(123)
```

That's a **remote invocation**.

5. Here's the easiest way to remember it 🧠

Ask:

"Who is calling the function?"

If it's:

```
Server code  
↓  
Server function
```



that's just a normal function invocation, even if the function is a Server Function.

No special network trip is needed.

But if:

```
Browser  
↓  
Server Function
```



then the browser can't directly execute it.

It invokes the **Server Reference**, which causes communication with the server.

6. This is the subtle point

A Server Function doesn't mean:

"Every time this function is called, a network request happens."



It means:

The function has semantics that allow it to be referenced and invoked from the client.

So:

```
</> TypeScript

'use server';

export async function addToCart(id) {
  ...
}
```

If server code does:

```
</> TypeScript

await addToCart(123);
```

that's simply:

```
Server
  ↓
normal function invocation
  ↓
addToCart()
```

But when the **client** invokes it through its **Server Reference**, then:

```
Client
  ↓
Server Reference
  ↓
network
  ↓
Server Function
```

7. One more example

Normal server-side helper

```
</> TypeScript

async function calculatePrice(productId) {
  const product = await db.products.find(productId);
```

```
    return product.price * 1.18;
  }
```

Used by:

```
</> TypeScript   
  
export default async function Page() {  
  const price = await calculatePrice(123);  
  
  return <div>{price}</div>;  
}
```

Flow:

```
Page()  
  ↓  
calculatePrice()  
  ↓  
DB 
```

No client involvement.

Server Function

```
</> TypeScript   
  
'use server';  
  
export async function purchase(productId) {  
  await db.orders.create({ productId });  
}
```

Client:

```
</> TypeScript   
  
'use client';  
  
import { purchase } from './actions';  
  
export function BuyButton({ productId }) {  
  return (  
    <button onClick={() => purchase(productId)}>  
      Buy  
    </button>  
  )  
}
```

```
);  
}
```

Conceptually:

```
CLICK  
↓  
purchase(123)  
↓  
Server Reference  
↓  
NETWORK  
↓  
Server  
↓  
actual purchase(123)  
↓  
DB
```



The interview distinction

If they ask:

"What's the difference between a server-side function and a Server Function?"

Say:

"A server-side function is simply a normal function that executes in the server environment. It can be called by other server code through an ordinary JavaScript function call. A Server Function has explicit Server Function semantics, typically through `'use server'`, which allows it to be represented by a Server Reference and invoked from client code through the server-action mechanism. The function implementation remains on the server."

And the one-liner:

```
Server-side function  
= where it executes
```

```
Server Function  
= special semantics for server invocation from the client
```



 That distinction is exactly what I want you to carry into 6.12.



ChatGPT can make mistakes. Check important info.